Supervised Learning Assignment #05 ID: 947129

1. Introduction

This report presents a comparative study of Convolutional Neural Network (CNN) architectures for image classification. The goal was to evaluate how different network parameters affect performance and to identify the best-performing configuration.

2. Methodology

I designed 10 different CNN configurations by varying:

- Number of convolutional layers (from shallow to deep)
- Number of filters (32 → 256)
- Dropout rates (0.2 – 0.5)
- Fully connected layer sizes
- Optimizers (Adam, SGD)
- Learning rates and weight decay

I also tried pretrained models with transfer learning and commented them the code part.

Data augmentation was applied using random flips, rotations, and normalization to improve generalization.

3. Experimental Setup

Each model was trained using:

- CrossEntropyLoss
- Early stopping
- Learning rate scheduling

Performance was evaluated on validation data using accuracy and loss.

Finally the best model is used for testing on unseen data.

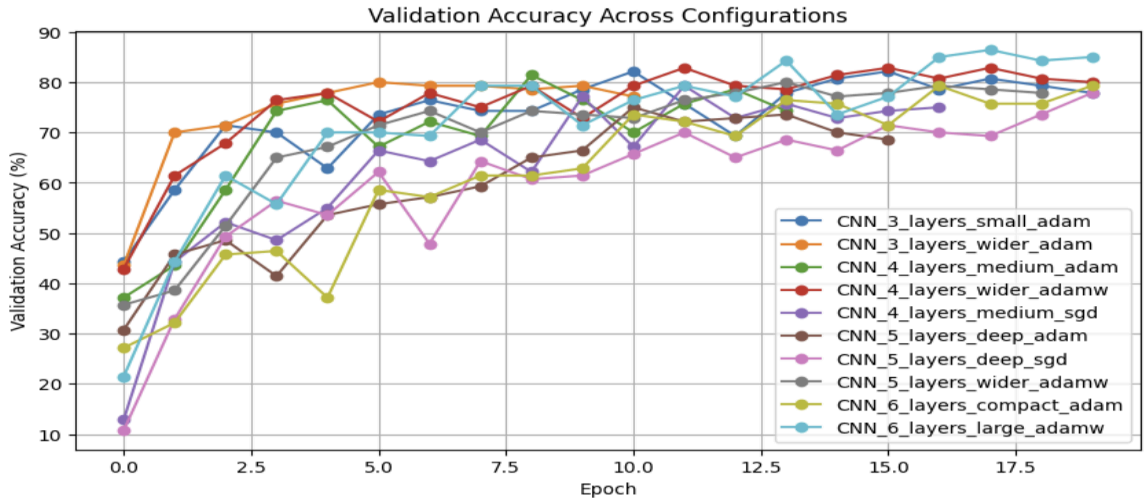
3. Results

Fig 1. Different varieties of CNN configurations and training result of each CNN

configurations

	name	conv_channels	dropout	fc_units	optimizer	lr	weight_decay
0	CNN_3_layers_small_adam	(32, 64, 128)	0.30	256	adam	0.0010	0.0001
1	CNN_3_layers_wider_adam	(64, 128, 256)	0.30	256	adam	0.0008	0.0001
2	CNN_4_layers_medium_adam	(32, 64, 128, 256)	0.35	256	adam	0.0008	0.0001
3	CNN_4_layers_wider_adamw	(64, 128, 256, 256)	0.40	256	adamw	0.0007	0.0005
4	CNN_4_layers_medium_sgd	(32, 64, 128, 256)	0.30	256	sgd	0.0100	0.0001
5	CNN_5_layers_deep_adam	(32, 64, 128, 256, 256)	0.40	256	adam	0.0007	0.0001
6	CNN_5_layers_deep_sgd	(32, 64, 128, 256, 256)	0.35	256	sgd	0.0100	0.0001
7	CNN_5_layers_wider_adamw	(64, 128, 256, 256, 512)	0.45	512	adamw	0.0005	0.0005
8	CNN_6_layers_compact_adam	(32, 64, 128, 128, 256, 256)	0.35	256	adam	0.0006	0.0001
9	CNN_6_layers_large_adamw	(64, 128, 128, 256, 256, 512)	0.45	512	adamw	0.0004	0.0007

	Rank	Model	Conv Layers	Conv Channels	Dropout	FC Units	Optimizer	Learning Rate	Weight Decay	Best Val Accuracy (%)	Best Val Loss	Epochs Ran
0	1	CNN_6_layers_large_adamw	11	(64, 128, 128, 256, 256, 512)	0.45	512	adamw	0.0004	0.0007	86.43	0.5132	20
1	2	CNN_4_layers_wider_adamw	7	(64, 128, 256, 256)	0.40	256	adamw	0.0007	0.0005	82.86	0.5719	20
2	3	CNN_3_layers_small_adam	5	(32, 64, 128)	0.30	256	adam	0.0010	0.0001	82.14	0.6550	20
3	4	CNN_4_layers_medium_adam	7	(32, 64, 128, 256)	0.35	256	adam	0.0008	0.0001	81.43	0.7561	14
4	5	CNN_3_layers_wider_adam	5	(64, 128, 256)	0.30	256	adam	0.0008	0.0001	80.00	0.6869	11
5	6	CNN_5_layers_wider_adamw	9	(64, 128, 256, 256, 512)	0.45	512	adamw	0.0005	0.0005	80.00	0.6448	19
6	7	CNN_4_layers_medium_sgd	7	(32, 64, 128, 256)	0.30	256	sgd	0.0100	0.0001	79.29	0.7090	17
7	8	CNN_6_layers_compact_adam	11	(32, 64, 128, 128, 256, 256)	0.35	256	adam	0.0006	0.0001	79.29	0.7368	20
8	9	CNN_5_layers_deep_sgd	9	(32, 64, 128, 256, 256)	0.35	256	sgd	0.0100	0.0001	77.86	0.7700	20
9	10	CNN_5_layers_deep_adam	9	(32, 64, 128, 256, 256)	0.40	256	adam	0.0007	0.0001	75.00	0.8966	16



Best configuration:

	Rank	Model	Conv Layers	Conv Channels	Dropout	FC Units	Optimizer	Learning Rate	Weight Decay	Best Val Accuracy (%)	Best Val Loss	Epochs Ran
0	1	CNN_6_layers_large_adamw	11	(64, 128, 128, 256, 256, 512)	0.45	512	adamw	0.0004	0.0007	86.43	0.5132	20

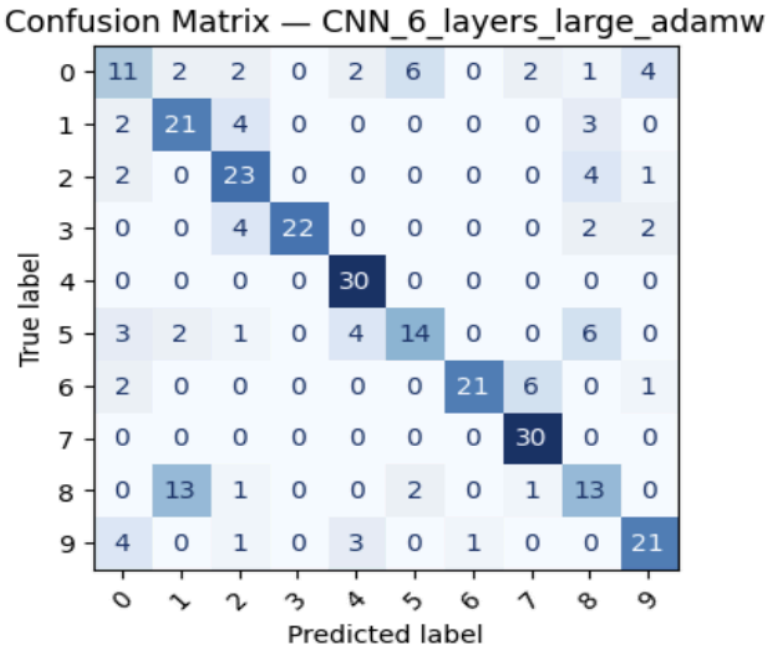
```
Best model: CNN_6_layers_large_adamw
Test Loss: 0.9401
Test Accuracy: 68.67%
```

Confusion matrix for the best model

]:

```
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)

fig, ax = plt.subplots(figsize=(10, 4))
disp.plot(ax=ax, cmap="Blues", xticks_rotation=45, colorbar=False)
plt.title(f"Confusion Matrix — {best_model_name}")
plt.show()
```



Classification report

```
report_dict = classification_report(y_true, y_pred, target_names=class_names, output_dict=True)
report_df = pd.DataFrame(report_dict).T
report_df
```

	precision	recall	f1-score	support
0	0.458333	0.366667	0.407407	30.000000
1	0.552632	0.700000	0.617647	30.000000
2	0.638889	0.766667	0.696970	30.000000
3	1.000000	0.733333	0.846154	30.000000
4	0.769231	1.000000	0.869565	30.000000
5	0.636364	0.466667	0.538462	30.000000
6	0.954545	0.700000	0.807692	30.000000
7	0.769231	1.000000	0.869565	30.000000
8	0.448276	0.433333	0.440678	30.000000
9	0.724138	0.700000	0.711864	30.000000
accuracy	0.686667	0.686667	0.686667	0.686667
macro avg	0.695164	0.686667	0.680600	300.000000
weighted avg	0.695164	0.686667	0.680600	300.000000

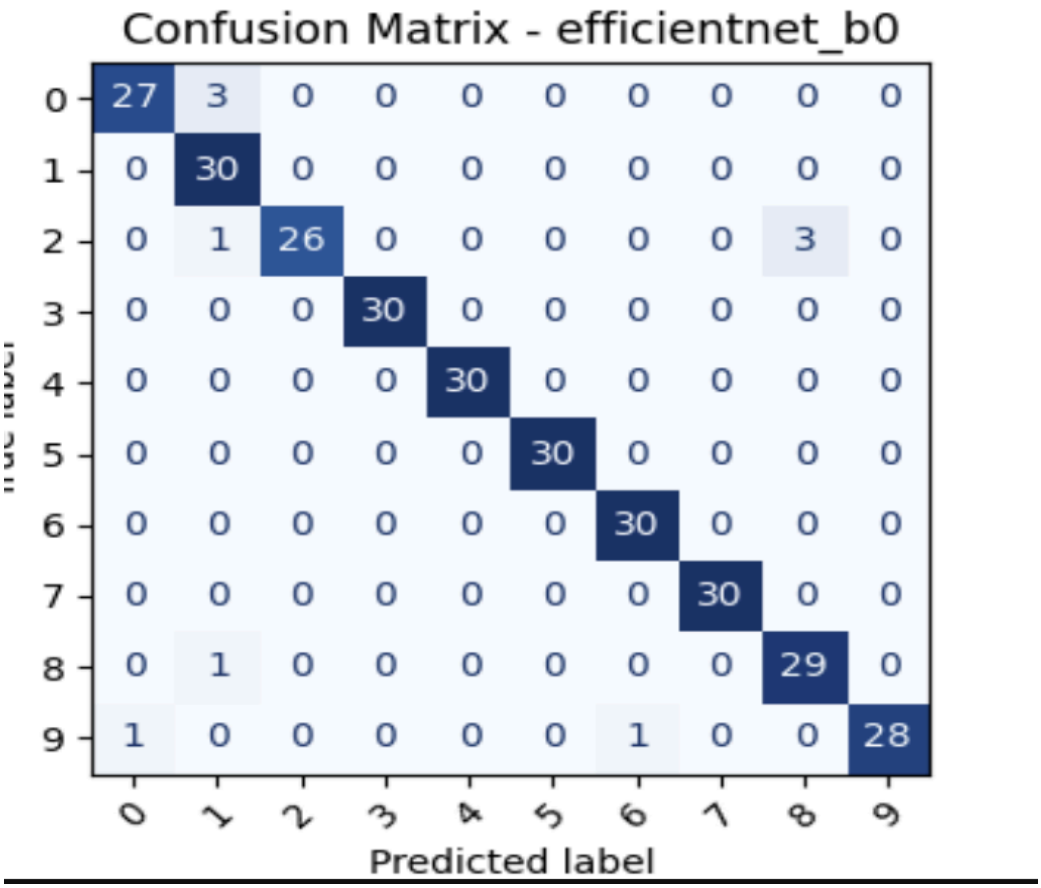
```
Number of displayed misclassified examples: 12
```

```
plt.figure(figsize=(16, 10))
for i, (img, true_label, pred_label) in enumerate(misclassified):
    plt.subplot(3, 4, i + 1)
    img = denormalize(img).clamp(0, 1)
    plt.imshow(np.transpose(img.numpy(), (1, 2, 0)))
    plt.title(f"True: {class_names[true_label]}\nPred: {class_names[pred_label]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

Classification results through Transfer Learning using two pretrained CNN models and one Custom CNN model

Model comparison table:

	model_name	best_val_acc	best_val_loss	test_acc	test_loss	train_time_sec	epochs_requested
0	efficientnet_b0	98.57	0.6189	96.67	0.6526	19.9139	20
1	resnet18	97.86	0.6164	95.67	0.6724	24.0843	20
2	custom_cnn	78.57	0.9189	73.33	1.0468	44.3759	20



Short analysis

- Models with **more CNN layers** usually learn richer features, but they may overfit if regularization is too weak.
- Increasing the **number of filters** increases model capacity and can improve accuracy, but also raises computation cost.
- **Batch normalization** stabilizes training and often improves convergence.
- **Dropout** helps reduce overfitting.
- **Adam / AdamW** often converge faster, while **SGD** may need more tuning and more epochs.
- The confusion matrix shows which classes are most often confused.
- The misclassified images help explain whether errors are caused by visual similarity, background clutter, poor lighting, or unusual viewpoints.